

Fraud Detection Deployment Guide

Local → Kubernetes → Dashboard

Step-by-Step with Expected Output at Every Stage

The Business Problem

Credit card fraud costs financial institutions billions annually. Traditional rule-based systems miss sophisticated fraud patterns. Machine learning models can analyze hundreds of data points per transaction to detect fraud that humans and simple rules would miss.

The Technology Stack (Plain Language)

What each tool does

Anaconda provides the trusted, pre-vetted software packages (like ingredients from a certified supplier). Metaflow is the workflow engine that organizes the ML pipeline into repeatable steps (like a recipe). Outerbounds is the production platform that runs those workflows on powerful cloud servers (like a commercial kitchen). Streamlit powers the interactive dashboard where the fraud ops team sees results.

Why Outerbounds Changes Everything

The comparison below shows the difference between building this system the traditional way versus using Outerbounds. The left column represents weeks of infrastructure work. The right column represents the same capability delivered in hours.

Workflow Stage	Without Outerbounds	With Outerbounds
Environment setup	Write Dockerfiles, configure Kubernetes, and manage cloud credentials manually	Run one command (outerbounds configure), paste a token, and everything is connected
Write ML code	Scattered Python scripts and notebooks with no standard structure or versioning	Metaflow flows: organized pipeline steps with automatic versioning and artifact storage
Train models	Run one configuration at a time on a single laptop. Hours per experiment.	4+ model configurations train simultaneously on separate cloud machines
Track experiments	Results in spreadsheets and scattered log files. Easy to lose.	Every run is automatically recorded with metrics, code, and results in one dashboard
Deploy to production	Build Docker images, CI/CD pipelines, Helm charts, and ingress rules. Weeks of DevOps.	One command: outerbounds app deploy. Auto-scaling, health checks, live URL.
Monitor + schedule	Set up Grafana for dashboards, Airflow for scheduling. Maintain both.	Built-in monitoring, deployment health, workflow scheduling, and cost reports
Reproduce results	Hunt down the code version, data snapshot, and environment. Often impossible.	Every run is stored in S3 with a unique ID. Reproduce any result with one API call.

The bottom line

What takes weeks of DevOps engineering without Outerbounds takes hours with it. Data scientists never touch Dockerfiles, Kubernetes configuration, or infrastructure setup. They focus entirely on building the fraud detection model.

Pipeline Overview: Three Connected Flows

Think of it like an assembly line

The fraud detection system is built as three connected stages, like stations on an assembly line. Stage 1 prepares the raw data. Stage 2 trains and selects the best model. Stage 3 scores new transactions. Each stage passes its output to the next, and every handoff is tracked automatically.

Stage 1: Data Preparation

284,807 real transactions

50K train / 10K test split

Steps: load data, add features, split into train/test sets

Stage 2: Model Training

4 model configurations tested

Best model selected by accuracy

Steps: train 4 models in parallel, pick the winner

Stage 3: Batch Scoring

100 transactions scored

APPROVE / REVIEW / BLOCK

Steps: load model, generate batch, score, classify

Each stage is called a "flow" in Metaflow. Flows are chained via run IDs, meaning each flow reads the output of the previous one. This creates a fully traceable pipeline: you can always trace a scoring decision back to the exact model and data that produced it.

Two Environments: Local vs. Kubernetes

Why two environments?

Developers first build and test on their laptop (local) for fast iteration. Once everything works, they deploy to Kubernetes (cloud) for production scale. **The key insight:** the code is identical. Only two flags change in the command. This is the core value of Outerbounds: same code, laptop to production.

Local Execution (Your Laptop)

- Runs on your laptop's CPU and RAM
- Loads real creditcard.csv data file (284K transactions)
- Artifacts stored on your local hard drive
- Packages from your conda environment
- Command: `python flows/X.py run`
- Best for: development, testing, fast iteration

Kubernetes Execution (Cloud)

- Runs on cloud machines (K8s pods)
- Uses synthetic data (real CSV isn't on the cloud)
- Artifacts stored in S3 cloud storage (shared, persistent)
- Packages installed automatically by Fast Bakery
- Command: `adds --environment=fast-bakery --with kubernetes`
- Best for: production, parallel training, deployment at scale

Important: separate datastores

Local runs and cloud runs use different storage systems. A cloud scoring flow cannot load a model from a local training run. When running the full pipeline on Kubernetes, all three stages must run on Kubernetes. Think of it like two separate filing cabinets: the laptop cabinet and the cloud cabinet. You can't pull a file from one and use it in the other.

Part A: Local Deployment (Your Laptop)

What you'll accomplish

By the end of this section, you will have run the entire fraud detection pipeline on your laptop: prepared data from 284,807 real transactions, trained 4 ML models in parallel, selected the best one, and scored 100 test transactions with APPROVE/REVIEW/BLOCK decisions. You'll also have a live dashboard running in your browser.

Prerequisites Checklist

Before starting, confirm you have:

- VS Code installed with the Outerbounds extension
- Miniconda installed (conda --version works)
- Git installed (git --version works)
- [fraud-detection-ob](#) repository cloned
- Conda environment created and activated:
 - `conda env create -f environment.yml`
 - `conda activate fraud-outerbounds`
- Use this link to configure local Workstations connection to OB
<https://ui.merced.obp.outerbounds.com/dashboard/configure/p/default>
- **Outerbounds configured:** outerbounds configure (token entered, outerbounds check shows 4 green)

Verify Project Structure

Run this to confirm all required files exist:

```
ls flows/ src/ data/ fraud-dash-outerbounds.py requirements.txt
```

You should see:

- flows/ — data_prep_flow.py, training_flow.py, scoring_flow.py, data_prep_flow_local.py, training_flow_local.py, scoring_flow_local.py
- src/ — config.py, models.py, data_utils.py, api_client.py, __init__.py
- data/ — creditcard.csv (download from Kaggle)

- fraud-dash-outerbounds.py, requirements.txt, deploy.sh in root

Why models and pickle files in your Outerbounds workflow → models/ — hybrid_fraud_*.pickle

The `models/` directory is only needed if you're running the original Jupyter notebook workflow, which manually trains a model and saves it as a `.pickle` file to disk. That's the older approach where the notebook handles everything — training, saving, and loading models by file path.

The Metaflow pipeline works differently. When the training flow saves `self.best_model`, Metaflow automatically stores it as a versioned artifact in its own datastore (local disk for local runs, S3 for Kubernetes runs). The scoring flow retrieves it by run ID — `Flow("FraudTrainingFlow")["284"]` — not by reading a file from the `models/` directory. So the Metaflow pipeline never writes to or reads from `models/` at all.

If you're only using the Metaflow flows, you can skip the notebook cell that creates the `models/` directory and saves `.pickle` files. If you're using both approaches (notebook for experimentation, Metaflow for production), then keep the `models/` directory for the notebook path.

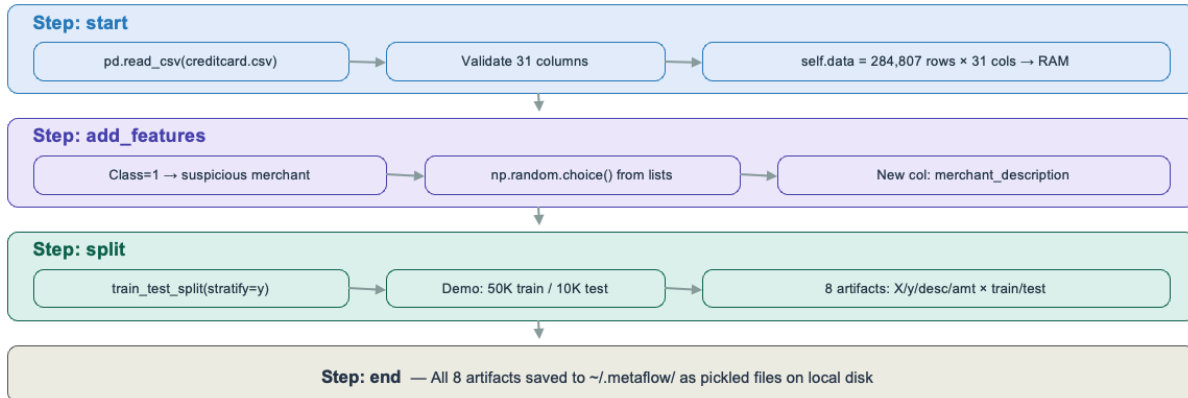
Step 1: Run Data Preparation

What is happening in this step

This step loads the real credit card transaction dataset (284,807 transactions, of which only 0.17% are fraudulent), adds merchant descriptions to make the data more realistic, and splits it into a training set (to teach the model) and a test set (to evaluate how well it learned). Think of it as sorting a warehouse of packages into a training pile and a testing pile.

Local: data_prep_flow.py — backend view

```
python flows/data_prep_flow.py run
```



Local data prep backend — Shows the 4 steps with the actual functions called: `pd.read_csv()` loading creditcard.csv, column validation, `np.random.choice()` for merchant names, `train_test_split()` with stratified sampling. All artifacts stay in RAM between steps and persist to `~/.metaflow/` on the local disk.

Command:

```
python flows/data_prep_flow_local.py run
```

What this does:

- Loads creditcard.csv (284,807 transactions) from data/ directory
- Validates all 31 required columns exist
- Adds synthetic merchant descriptions (legitimate vs. suspicious names)
- Splits into 50,000 train / 10,000 test sets (demo mode)
- Stores all artifacts in local Metaflow datastore

Expected terminal output:

```
Workflow starting (run-id XXX)
Dataset loaded: 284,807 transactions
Fraud rate: 0.1727%
Training: 50,000 transactions
Testing: 10,000 transactions
Data Prep Complete
```

Next: `python flows/training_flow.py run --data_run_id XXX`

IMPORTANT

Write down the run ID number (e.g., 148). You need it for the next step. The run ID is printed in the output after 'Workflow starting (run-id XXX)' and again in the 'Next:' command at the end.

VERIFY IN OUTERBOUNDS UI: Components > Runs > FraudDataPrepFlow

4 green steps in the DAG: start > add_features > split > end. Click any step to see logs. The run has a laptop icon (not cloud), indicating local execution.

Step 2: Run Model Training

What is happening in this step

This step takes the prepared data and trains 4 different versions of a fraud detection model, each with slightly different settings (called hyperparameters). It's like testing 4 different recipes to see which one produces the best results. The system then picks the winner based on accuracy (F1 score). All 4 trains simultaneously on your laptop.

Local: training_flow.py — backend view

```
python flows/training_flow.py run --data_run_id 148
```

Step: start — Flow("FraudDataPrepFlow")["148"] → loads 8 artifacts from disk → defines 4 HP configs

foreach="hp_grid" triggers 4 parallel processes on your laptop CPU

Train 1

100 trees, depth 12

```
RandomForestClassifier
.fit(X_train, y_train)
F1=0.8421, AUC=0.9631
```

Train 2

200 trees, depth 10

```
RandomForestClassifier
.fit(X_train, y_train)
F1=0.8421, AUC=0.9631
```

Train 3

200 trees, depth 15

```
RandomForestClassifier
.fit(X_train, y_train)
F1=0.8421, AUC=0.9631
```

Train 4

300 trees, depth 12

```
RandomForestClassifier
.fit(X_train, y_train)
F1=0.8421, AUC=0.9631
```

Step: select_best

max(inputs, key=F1) → stores best_model as plain sklearn RandomForestClassifier

Step: end — Model + metrics saved to `~/metaflow/`

Key: model stored as plain sklearn — not `OptimizedHybridDetector` — so K8s pods can unpickle it

Local training backend — Shows the `foreach` fan-out creating 4 parallel Python processes on your laptop CPU, each calling `RandomForestClassifier.fit()` with different hyperparameters. The `select_best` step picks `max(F1)` and stores the plain sklearn model (not the custom wrapper class).

Command (replace XXX with your data prep run ID):

```
python flows/training_flow_local.py run --data_run_id <DATA_RUN_ID>
```

What this does:

- Loads the train/test data from the data prep run
- Creates 4 RandomForest models with different hyperparameters
- Runs all 4 in parallel using Python multiprocessing
- Evaluates each on the test set (precision, recall, F1, AUC)
- Selects the best model by F1 score
- Stores the plain sklearn RandomForestClassifier as an artifact

Expected terminal output:

```
Foreach yields 4 child steps
Training: {'n_estimators': 100, 'max_depth': 12}
Training: {'n_estimators': 200, 'max_depth': 10}
Training: {'n_estimators': 200, 'max_depth': 15}
Training: {'n_estimators': 300, 'max_depth': 12}
F1=0.8421, AUC=0.9631
=====
HYPERPARAMETER SEARCH RESULTS
=====
{'n_estimators': 100, 'max_depth': 12} → F1=0.8421, AUC=0.9631 ← BEST
Training Complete
Next: python flows/scoring_flow.py run --training_run_id YYY
```

Write down the training run ID for the next step.

VERIFY IN OUTERBOUNDS UI: Components > Runs > FraudTrainingFlow

Fan-out DAG pattern: start > [4 parallel train boxes] > select_best > end. Each train box shows its own F1 and AUC metrics. One is marked BEST.

Step 3: Run Batch Scoring

What is happening in this step

This is the moment of truth: the trained model scores 100 test transactions and decides which to approve, flag for review, or block. It's like a security guard examining each person at a checkpoint and deciding who passes (APPROVE), who gets additional screening (REVIEW), and who gets stopped (BLOCK).

Local: scoring_flow.py — backend view

```
python flows/scoring_flow.py run --training_run_id 150
```

Step: start — Flow("FraudTrainingFlow")["150"] → loads best_model, best_metrics, model_config

Step: load_transactions

Generate 90 legit (variance 0.5)

Generate 10 fraud (variance 3.0)

DataFrame: 100 rows × 30 features

Step: score

predict_proba()
on 100 rows

< 0.5

APPROVE (90)

0.5 – 0.8

REVIEW (4)

≥ 0.8

BLOCK (6)

Step: summarize (@card)

Total 100 | Approved 90 | Review 4 | Blocked 6 | Accuracy 1.0 | All 10 frauds flagged

Step: end — Results to ~/.metaflow/. @card report viewable in Outerbounds UI.

Local scoring backend — Traces data from synthetic generation (90 legit at 0.5 variance, 10 fraud at 3.0 variance) through `predict_proba()`, through the three threshold gates (< 0.5, 0.5–0.8, > 0.8), to the APPROVE/REVIEW/BLOCK output counts.

Command (replace YYY with your training run ID):

```
python flows/scoring_flow_local.py run --training_run_id <TRAINING_RUN_ID>
```

What this does:

- Loads the best-trained model from the training run
- Generates 100 test transactions (90 legitimate + 10 suspicious)
- Scores each transaction with the model
- Classifies as APPROVE (score < 0.5), REVIEW (0.5–0.8), or BLOCK (> 0.8)
- Prints a batch summary with flagged transactions

Expected terminal output:

You should get something like the below as a result (the values will differ)

```
Loaded model from run YYY: RandomForestClassifier
```

```
Batch: 100 transactions (90 legit, 10 suspicious)
```

```
=====
```

```
BATCH SUMMARY
```

```
=====
```

```
Total: 100
```

```
Approved: 90
```

```
Review: 4
```

```
Blocked: 6
```

```
Accuracy: 1.0000
```

```
True frauds: 10, Flagged: 10
```

```
Flagged:
```

```
UNKNOWN MERCHANT 9991 $539.14 0.8300 BLOCK 1
```

```
BITCOIN ATM UNKNOWN $460.62 0.8400 BLOCK 1
```

```
...
```

VERIFY IN OUTERBOUNDS UI: Components > Runs > FraudScoringFlow

All steps green. Click the summarize step to see the BATCH SUMMARY and flagged transactions in the logs.

LOCAL DEPLOYMENT COMPLETE

You have successfully run all three flows locally. The pipeline loaded real data (creditcard.csv), trained 4 models in parallel, selected the best, and scored transactions with APPROVE/REVIEW/BLOCK decisions. All runs are visible in the Outerbounds UI under Runs.

Step 4: Run the Streamlit Dashboard

What is happening in this step

The dashboard is the interface that the fraud operations team uses daily. It provides real-time metrics, the ability to test individual transactions, analytics on fraud patterns, and system health monitoring. Think of it as the control room for the fraud detection system.

Command:

```
streamlit run fraud-dash-outerbounds.py
```

This opens the dashboard in your browser at <http://localhost:8501> with 4 pages:

- Dashboard — Real-time metrics, transaction volume charts, live feed
- Test Transaction — Interactive fraud testing with APPROVE/REVIEW/BLOCK results
- Analytics — Fraud distribution, trends, confusion matrix, business impact
- System Status — API connectivity tests, endpoint configuration, diagnostics

Press Ctrl+C in the terminal to stop the dashboard.

Local deployment complete

You have now run the entire fraud detection pipeline locally: prepared real data, trained 4 models in parallel, selected the best, scored transactions with APPROVE/REVIEW/BLOCK decisions, and viewed results in a live dashboard. All runs are visible in the Outerbounds UI. Next: deploying the same pipeline to the cloud.

Key Differences: Local vs. Kubernetes

Aspect	Local	Kubernetes
Command	python flows/X.py run	python flows/X.py --environment=fast-bakery --with kubernetes run
Runs on	Your laptop CPU/RAM	Outerbounds K8s pods (cloud VMs)
Data source	data/creditcard.csv (local file)	Synthetic data (CSV not on pod)
Artifacts stored	~/metaflow/ (local disk)	S3 bucket (shared, persistent)
Packages	From your conda env	Installed by @pypi decorator + Fast Bakery
src/ imports	Work (local filesystem)	DO NOT work (not on pod). All logic must be inline.
Run IDs	Stored locally. Only visible to local runs.	Stored in S3. Only visible to K8s runs.
UI icon	Laptop icon	Cloud icon with pod names in logs

Speed	Depends on your laptop	Scales to cluster resources. Parallel tasks get separate pods.
--------------	------------------------	---

CRITICAL RULE

Local runs and K8s runs use DIFFERENT datastores. A K8s scoring flow CANNOT load a model from a local training run (and vice versa). When chaining flows on K8s, ALL flows in the chain must run on K8s.

Part B: Kubernetes Deployment (Cloud)

What changes and what stays the same

The code is identical. The ML logic doesn't change. What changes: the command adds two flags (`--environment=fast-bakery --with kubernetes`), execution moves from your laptop to cloud machines, and data/models are stored in cloud storage (S3) instead of your hard drive. The entire point of Outerbounds is to make this transition seamless: same code, laptop to production, no DevOps required.

Additional Requirements for Kubernetes

Three requirements for Kubernetes execution

1. @pypi decorator on every step

K8s pods start with minimal Python — no pandas, numpy, or sklearn.

Use Python 3.9-compatible versions: pandas 2.0.3, numpy 1.24.3, sklearn 1.3.2

Without: `ModuleNotFoundError: pandas`

With: `@pypi(packages=DEPS)` on each step

2. No imports from src/

The pod does not have your local project structure.

All logic must be inline in the flow file — no `from src.X import Y`.

Without: `ModuleNotFoundError: src`

With: `import pandas as pd (inline)`

3. --environment=fast-bakery flag

Tells Outerbounds to build the container image with your @pypi packages.

`--environment=conda` fails (Anaconda channel 403). Use `fast-bakery`.

Without: `@pypi` requires `--environment`

With: `--environment=fast-bakery`

All three are needed — missing any one causes the K8s run to fail

Your flows need three modifications to work on K8s:

Requirement 1: @pypi Decorator

K8s pods start with a minimal Python image (no pandas, numpy, sklearn). You must declare dependencies:

```
from metaflow import FlowSpec, step, pypi

DEPS = {"pandas": "2.0.3", "numpy": "1.24.3", "scikit-learn": "1.3.2"}

class MyFlow(FlowSpec):
    @pypi(packages=DEPS)
    @step
    def start(self):
        import pandas as pd # now available
```

Note: Use versions compatible with Python 3.11.15 (the base image version). numpy >= 2.0 requires Python 3.11 and will fail.

Requirement 2: No src/ Imports

The K8s pod does not have your project's src/ directory. All logic must be inline in the flow file:

```
# WRONG - fails on K8s:
from src.data_utils import load_fraud_data

# CORRECT - works everywhere:
import pandas as pd
data = pd.read_csv('data/creditcard.csv')
```

Requirement 3: --environment=fast-bakery Flag

This tells Outerbounds to use Fast Bakery to build a container image with your @pypi packages:

```
python flows/X.py --environment=fast-bakery --with kubernetes run
```

Step 1: Data Preparation on Kubernetes

Command

```
python flows/data_prep_flow.py --environment=fast-bakery --with kubernetes run
```

What happens behind the scenes

1. Fast Bakery builds a container image with your packages (2-18 seconds).
2. Your code is uploaded to S3 cloud storage.
3. A cloud machine (pod) starts and downloads your code.
4. Since the real CSV file isn't available on the cloud, synthetic data is generated automatically.
5. Prepared data is stored in S3 for the next stage.
6. The process repeats for each step, with a fresh cloud machine for each one. Key insight: 4 separate cloud machines run, each starting fresh. Data passes through S3 between them, not through memory.

K8s: data_prep_flow.py — backend view

```
python flows/data_prep_flow.py --environment=fast-bakery --with kubernetes run
```

Fast Bakery: builds image — Python 3.9 + pandas 2.0.3 + numpy 1.24.3 + sklearn 1.3.2 (~2-18s)

Pod 1: start (t-ddd54e1d)

Pull baked image
from registry

Download code
pkg from S3

CSV not found!
Generate 50,087 synthetic

Pod 2: add_features (t-c9abea05)

Download data from S3

Add merchant names

Save to S3

Pod 3: split (t-39bb5162)

Download data

train_test_split()

Save 8 artifacts

Pod 4: end (t-12d18b47) — Downloads artifacts, prints summary, generates @card

S3 bucket

(shared store)

self.data
(pickled)

updated data

X_train, X_test
y_train, y_test
desc, amt *2

Key: 4 separate pods, each starts fresh. Data passes through S3, not RAM. No local CSV.

K8s data prep backend — Shows 4 separate pods, each pulling the baked image, downloading the code package from S3, and passing **self.data** through S3 between pods. Highlights that CSV doesn't exist on pods so synthetic data is generated.

Expected terminal output:

You should get something like the below as a result (the values will differ)

Baking [#01] ... Python: None, pandas: 2.0.3, numpy: 1.24.3, scikit-learn: 1.3.2

Baked [#01] in 1.72 seconds!

Workflow starting (run-id 182)

[pod t-ddd54e1d-jntlh-vsmwr] Setting up task environment.

[pod t-ddd54e1d-jntlh-vsmwr] Downloading code package...

```
[pod t-ddd54e1d-jntlh-vsmwr] CSV not found - generating synthetic dataset for K8s  
[pod t-ddd54e1d-jntlh-vsmwr] Synthetic dataset: 50,087 transactions  
Done! See the run in the UI at https://ui.merced.obp.outerbounds.com/...
```

Write down the run ID.

VERIFY IN OUTERBOUNDS UI: Components > Runs > FraudDataPrepFlow (run with cloud icon)

4 green steps, each showing pod names like t-ddd54e1d-jntlh-vsmwr. The code package and artifacts are stored in S3.

Step 2: Model Training on Kubernetes

Command:

```
python flows/training_flow.py --environment=fast-bakery --with kubernetes run --data_run_id XXX
```

The parallel advantage

This is where cloud deployment shines. On your laptop, the 4 model configurations share one CPU. On Kubernetes, each configuration gets its own dedicated cloud machine (pod) with its own CPU and RAM. All 4 train simultaneously. The total: 7 pods (1 start + 4 parallel trains + 1 selection + 1 end). Scaling from 4 to 40 or 400 configurations is the same pattern: just expand the list. Outerbounds schedules the pods.

K8s: training_flow.py — backend view

--environment=fast-bakery --with kubernetes run --data_run_id 182

Fast Bakery: reuses cached image (~2s) — pandas 2.0.3 + numpy 1.24.3 + sklearn 1.3.2

Pod: start (t-09d63efe) — reads 8 artifacts from S3 (run 182) → 4 HP configs

4 pods scheduled simultaneously on K8s cluster

Pod: train/1010
t-51f5926b
100 trees, depth 12
Downloads X_train from S3
.fit() → F1=1.0
Saves model to S3

Pod: train/1011
t-b9aa7e04
200 trees, depth 10
Downloads X_train from S3
.fit() → F1=1.0
Saves model to S3

Pod: train/1012
t-c2d3c207
200 trees, depth 15
Downloads X_train from S3
.fit() → F1=1.0
Saves model to S3

Pod: train/1013
t-49140081
300 trees, depth 12
Downloads X_train from S3
.fit() → F1=1.0
Saves model to S3

Pod: select_best (t-260ab2cd)
Downloads all 4 models + metrics → picks max(F1) → stores best_model as plain sklearn

Pod: end (t-c5658638) — All artifacts in S3 under run 183

7 pods total: 1 start + 4 parallel trains + 1 select + 1 end

K8s training backend — Shows the actual pod names from your run (t-51f5926b, t-b9aa7e04, etc.), the S3 artifact download/upload cycle for each pod, and the 7-pod total (1 start + 4 train + 1 select + 1 end)

What happens:

1. Fast Bakery reuses the cached image (bakes in ~2 seconds)
2. Start step pod loads data artifacts from S3 (from the data prep run)
3. 4 separate pods spin up simultaneously for parallel training
4. Each pod trains a RandomForest with different hyperparameters
5. Results merge in the select_best pod
6. Best model stored as a plain sklearn RandomForestClassifier in S3

Expected terminal output:

You should get something like the below as a result (the values will differ)

Baked [#01] in 1.72 seconds!

Foreach yields 4 child steps

[pod t-51f5926b] Training: {'n_estimators': 100, 'max_depth': 12}

[pod t-b9aa7e04] Training: {'n_estimators': 200, 'max_depth': 10}

[pod t-c2d3c207] Training: {'n_estimators': 200, 'max_depth': 15}

[pod t-49140081] Training: {'n_estimators': 300, 'max_depth': 12}


```
[pod t-51f5926b] F1=1.0, AUC=1.0  
HYPERPARAMETER SEARCH RESULTS  
{'n_estimators': 100, 'max_depth': 12} → F1=1.0, AUC=1.0 ← BEST  
Training Complete: F1=1.0, AUC=1.0
```

VERIFY IN OUTERBOUNDS UI: [Components](#) > [Runs](#) > [FraudTrainingFlow](#)

Fan-out DAG with cloud icon. The 4 parallel train steps each show different pod names. All green.

Step 3: Batch Scoring on Kubernetes

Command

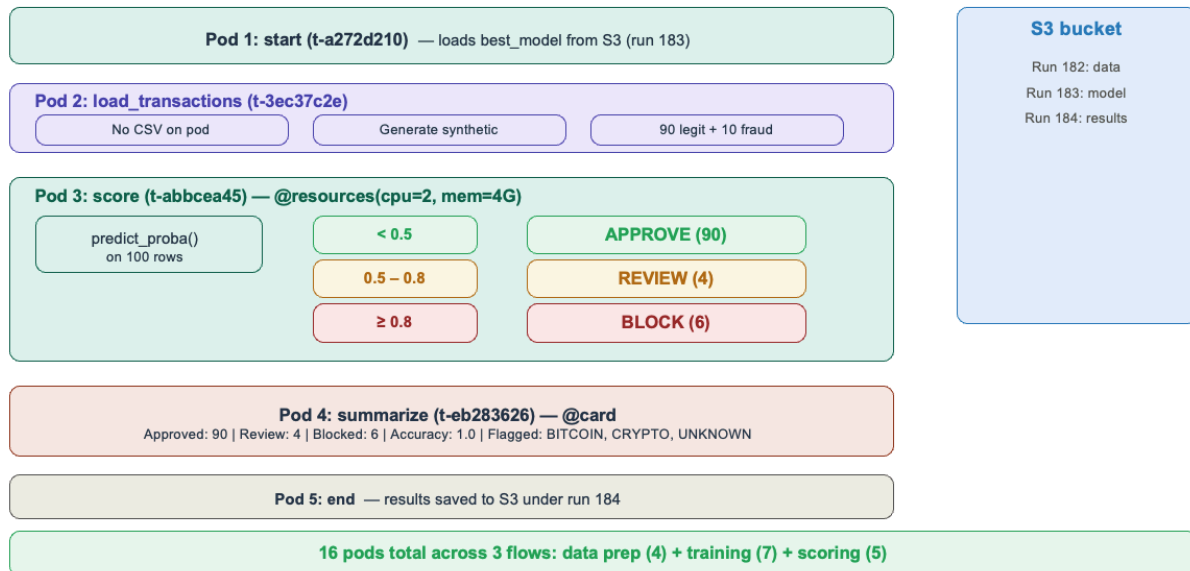
```
python flows/scoring_flow.py --environment=fast-bakery --with kubernetes run --training_run_id  
YYY
```

Cross-run lineage

The scoring flow reaches into S3 and loads the best model from the training run (run 183) into a brand new pod. This is full reproducibility in action: Run 182 has the data, Run 183 has the model, Run 184 has the scoring results. You can trace any decision back to the exact model and data that produced it, even months later.

K8s: scoring_flow.py — backend view

--environment=fast-bakery --with kubernetes run --training_run_id 183



Expected terminal output:

You should get something like the below as a result (the values will differ)

Loaded model from run YYY: RandomForestClassifier

Batch: 100 transactions (90 legit, 10 suspicious)

=====

BATCH SUMMARY

=====

Total: 100, Approved: 90, Review: 4, Blocked: 6

Accuracy: 1.0000, True frauds: 10, Flagged: 10

Flagged:

UNKNOWN MERCHANT 9991 \$539.14 0.8300 BLOCK 1

BITCOIN ATM UNKNOWN \$460.62 0.8400 BLOCK 1

CRYPTO EXCHANGE UNVERIFIED \$949.06 0.8000 REVIEW 1

KUBERNETES DEPLOYMENT COMPLETE

The full pipeline ran end-to-end on Kubernetes. Data was generated on pods, 4 models trained in parallel across 4 pods, the best model selected, and 100 transactions scored with APPROVE/REVIEW/BLOCK decisions. All artifacts are stored in S3. All runs are visible in the Outerbounds UI with cloud icons.

Part C: Deploy the Dashboard to Outerbounds

What this achieves

This step takes the dashboard that was running on your laptop (localhost:8501) and deploys it as a permanent, publicly accessible web application. Anyone on the team can open the URL in a browser to view fraud detection metrics, test transactions, and review analytics. It runs 24/7 with auto-scaling: if traffic increases, Outerbounds automatically adds more workers.

How the deploy script works

The entire deployment is handled by a single shell script (`deploy.sh`) that generates the correct `requirements.txt`, validates your project structure, and runs the Outerbounds deploy command. No manual file copying or editing required.

Step 1: Verify project structure

```
fraud-dash-outerbounds/
  app.py          # Dashboard entrypoint (Streamlit app)
  deploy.sh       # Deploy script (Deploys to Outerbounds)
  src/
    __init__.py   # Imports config, data_utils, models, api_client
    config.py     # Endpoints, API token, merchant lists
    data_utils.py # Data loading utilities (imports sklearn)
    models.py     # Model classes (imports sklearn, requests)
    api_client.py # API client (imports requests)
```

Step 2: Run the deploy

```
./deploy.sh
```

Step 3: What the deploy script does

Run this single command (it handles everything):

1. **Validates** that `app.py` and `src/config.py` exist in the current directory
2. **Generates** `requirements.txt` with only the 6 packages the dashboard actually imports at runtime
3. **Packages the code** using `--package-src-path`. (the entire project directory)
4. **Includes only Python files** via `--package-suffixes py` (`app.py` and all `src/*.py`)
5. **Builds a container image** using Fast Bakery with streamlit, pandas, numpy, plotly, requests, and scikit-learn

6. **Uploads the code package** to S3
7. **Starts a worker pod** that runs the Streamlit command
8. **Applies the filename workaround:** the `bash -c` rename loop fixes a known Outerbounds issue where the code packager strips dots from filenames (app.py becomes apppy)
9. **Returns a public URL** once the app is healthy

Expected terminal output:

You should get something like the below as a result (the values will differ)

```
=====
Fraud Dashboard - Outerbounds Deploy
=====

Project directory: /Users/you/fraud-dash-outerbounds

Created requirements.txt:
streamlit
pandas
numpy
plotly
requests
scikit-learn

Deploying fraud-dashboard to Outerbounds...

Baking [fraud-dashboard] ...
Baked [fraud-dashboard] in 19.01 seconds!
Code package saved to s3://...
Deploying app to the platform...
First worker came online
All workers are now running
App fraud-dashboard deployed! App available on URL: https://ui-c-XXXXX.merced.obp.outerbounds.com

=====
Deploy complete
=====
```

YOUR DASHBOARD IS LIVE

Open the URL in any browser. Share it with your team. It runs 24/7 on the Outerbounds platform with auto-scaling.

VERIFY IN OUTERBOUNDS UI: Components > Deployments

The Fraud Dashboard appears with a green status, URL, worker count (1-2), a requests/minute graph, and an autoscaling chart.

Step 5: Managing the Deployed Dashboard

Task	Command
View all deployed apps	<code>outerbounds app list</code>
Check app logs	<code>outerbounds app logs --name fraud-dashboard</code>
Redeploy with updates	<code>./deploy.sh</code> command with <code>--force-upgrade</code> added
Delete the app	<code>outerbounds app delete --name fraud-dashboard</code>
Deploy with new name	Change <code>--app</code> name at the top of <code>deploy.sh</code>

Part D: Troubleshooting Guide

Local Deployment Issues

Error	Cause	Fix
metaflow_token FAIL	outerbounds/metaflow not installed in conda env	<code>pip install outerbounds metaflow</code> , then <code>outerbounds configure</code>
kubeconfig FAIL	Token expired	Re-run <code>outerbounds configure</code> with fresh token
FileNotFoundError: creditcard.csv	Not in <code>data/</code> directory	Download from Kaggle and place in <code>data/creditcard.csv</code>
flows/ not found	Directory not created	<code>mkdir flows</code> and copy flow files
All 100 transactions APPROVED	Test batch had no fraud cases	Expected with real CSV <code>tail(100)</code> . Use synthetic batch instead.

Kubernetes Deployment Issues

Error	Cause	Fix
No module named 'src'	Flow imports from src/ which is not on the K8s pod	Use the inline flow versions with no src imports
No module named 'pandas'	Pod base image has no pandas	Add @pypi(packages=DEPS) decorator to every step
@pypi requires --environment	Metaflow needs to know which env system to use	Add --environment=fast-bakery to the command
numpy requires Python>=3.11	Base image is Python 3.9, numpy 2.x needs 3.11	Use numpy 1.24.3 and pandas 2.0.3 (compatible with 3.9)
KeyError: 'XXX' (run ID not found)	Training ran locally but scoring runs on K8s (different datastores)	Run ALL flows in the chain on K8s. Local and K8s runs cannot cross-reference.
Pickled model references custom class	Model was saved as OptimizedHybridDetector, not available on pod	Use training_flow.py that stores plain sklearn RandomForestClassifier
micromamba channel 403 error	@pypi with --environment=pypi tries to use local micromamba	Use --environment=fast-bakery instead (builds on Outerbounds infra)

Dashboard Deployment Issues

Error	Cause	Fix
ModuleNotFoundError: No module named 'X'	A transitive import is missing from requirements.txt	Run the <code>grep</code> audit command above and add the missing package <code>grep -r "^import ^from" app.py src/*.py</code>
File does not exist: app.py	Code packager stripped dots from filenames	The <code>bash -c</code> rename workaround in <code>deploy.sh</code> handles this automatically
ERROR: app.py not found	Running <code>deploy.sh</code> from the wrong directory	<code>cd</code> into the project root first
Old deployment is still running	Previous config is cached	<code>outerbounds app delete --name fraud-dashboard</code> , then <code>./deploy.sh</code>

Worker crashlooping	Import error or missing config	Check logs: <code>outerbounds app logs --name fraud-dashboard</code>
----------------------------	--------------------------------	--

Part E: Complete Command Reference

Local Commands

Task	Command
Activate environment	<code>conda activate fraud-dash-outerbounds</code>
Configure Outerbounds	<code>outerbounds configure</code>
Verify connection	<code>outerbounds check</code>
Data prep (local)	<code>python flows/data_prep_flow.py run</code>
Training (local)	<code>python flows/training_flow.py run --data_run_id <ID></code>
Scoring (local)	<code>python flows/scoring_flow.py run --training_run_id <ID></code>
Dashboard (local)	<code>streamlit run fraud-dashboard.py</code>

Kubernetes Commands

Task	Command
Data prep (K8s)	<code>python flows/data_prep_flow.py --environment=fast-bakery --with kubernetes run</code>
Training (K8s)	<code>python flows/training_flow.py --environment=fast-bakery --with kubernetes run --data_run_id <ID></code>

Scoring (K8s)	python flows/scoring_flow.py --environment=fast-bakery --with kubernetes run --training_run_id <ID>
---------------	---

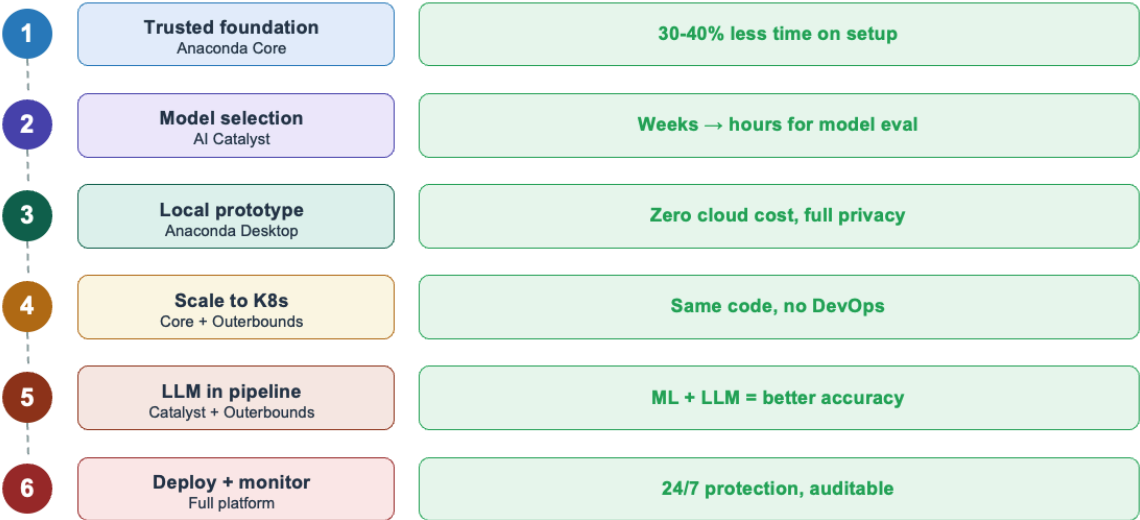
Dashboard Deployment Commands

Task	Command
Deploy dashboard	See Part C3 for full command
List apps	outerbounds app list
View logs	outerbounds app logs --name fraud-dashboard
Force redeploy	Add --force-upgrade to deploy command
Delete app	outerbounds app delete --name fraud-dashboard

THIS IS A DRAFT I AM WOKING WITH THE TEAM ON

Developer Journey

The complete journey — idea to production



Together: secure packages → governed models → local prototype → production scale

Stage 1 Anaconda Core — set up a trusted foundation

What the developer does:

Installs Anaconda Distribution — gets Python, conda, Jupyter, and 300+ validated packages in one download. Creates a conda environment for the fraud detection project. Every package is built, tested, and signed by Anaconda engineers.

Automated dependency resolution prevents the "works on my machine" problem. Security assessments are included — no manual CVE scanning needed.

Without Anaconda Core Days resolving package conflicts Manual security vetting of every library Different versions across team machines	With Anaconda Core One install, ready in minutes Pre-assessed security for all packages Identical environments everywhere
---	---

Customer value: Eliminates 30-40% of time wasted on dependency conflicts and security vetting

In our fraud project:
conda activate fraud-dash-outerbounds
pip install outerbounds metaflow scikit-learn pandas numpy streamlit plotly xgboost
outerbounds configure → token entered → 4 green checks → ready to build

Stage 2

AI Catalyst — discover and select governed AI models

What the developer does:

Browses 50+ curated, benchmarked open-source LLMs (Llama, Nemotron, Mistral) in AI Catalyst's model catalog. Each model has an AI Bill of Materials, OWASP/MITRE risk scores, and performance benchmarks. Filters by use case, license, parameter count, security score. Compares models side-by-side. Selects Meta-Llama-3.1-8B-Instruct for fraud analysis.

Without AI Catalyst

Weeks sifting through 1.7M models
No security visibility or risk scoring
Shadow AI — ungoverned model usage

With AI Catalyst

Pre-vetted catalog, benchmarks included
OWASP + MITRE risk scores per model
Full audit trail for compliance reviews

Customer value: Model selection from weeks to hours. Compliance-ready from day one.

In our fraud project:

Selected Meta-Llama-3.1-8B-Instruct via AI Catalyst for second-opinion fraud analysis.
ML model flags suspicious transaction → LLM reviews merchant + amount → combined score.
Model accessed via Anaconda Connect API with API key — governed, tracked, auditable.

Stage 3

Anaconda Desktop — prototype locally, no cloud needed

What the developer does:

Downloads AI Catalyst models to their laptop via Anaconda Desktop. Runs LLM inference locally — data never leaves the device. Manages conda environments, Jupyter notebooks, and model API endpoints from one unified app. Builds the full fraud pipeline (data prep → training → scoring) and tests with real creditcard.csv data.

Without Anaconda Desktop

Cloud costs from day one of prototyping
Sensitive data sent to external APIs
Juggling 5+ tools (conda, Jupyter, model server...)

With Anaconda Desktop

Zero cloud cost for experimentation
Full data privacy — nothing leaves device
One app for envs, models, notebooks, API

Customer value: Zero cloud cost for experimentation. Full data privacy. Fast iteration cycles.

In our fraud project:

Ran all 3 flows locally: data prep (284K rows) → training (4 models, $F1=0.84$) → scoring (100 txns).
Tested Streamlit dashboard at localhost:8501. Iterated on model thresholds.
All runs visible in Outerbounds UI under Runs with laptop icon. Zero cloud spend.

Stage 4

Anaconda Core + Outerbounds — scale to production

What the developer does:

Takes the exact same code from their laptop and adds two flags: `--environment=fast-bakery --with kubernetes`. Outerbounds handles everything: Fast Bakery builds a Docker image with Anaconda Core packages, uploads code to S3, schedules K8s pods, parallelizes training across 4 pods, stores artifacts in S3, and provides a full UI for monitoring.

Without Outerbounds

Write Dockerfiles, Helm charts, K8s YAML
Build CI/CD pipeline from scratch
Weeks of DevOps engineering

With Outerbounds

Same code, add two flags
Fast Bakery + S3 + K8s — all managed
Hours to production, not weeks

Customer value: Same code, laptop to production. No Dockerfiles, no DevOps, no YAML.

In our fraud project:

Data prep: 4 K8s pods, synthetic data (run 182). Training: 7 pods, 4 parallel models, F1=1.0 (run 183).
Scoring: 5 pods, 100 txns scored — 90 approved, 4 review, 6 blocked (run 184).
Total: 16 pods across 3 flows. All visible in Outerbounds UI with cloud icons.

Stage 5

AI Catalyst + Outerbounds — LLM-enhanced pipeline

What the developer does:

The fraud scoring flow calls an AI Catalyst model (Llama 3.1 8B) via the Anaconda Connect API. The ML model flags a suspicious transaction → the LLM reviews the merchant description and amount → a combined weighted score determines APPROVE, REVIEW, or BLOCK. This hybrid approach catches fraud that statistical models alone would miss.



Customer value: LLM reasoning + ML precision = fewer false positives, more fraud caught.

In our fraud project:

Hybrid scoring: XGB weight 0.6 + LLM weight 0.4. LLM threshold > 0.3 triggers AI review.
APPROVE < 0.5 | REVIEW 0.5–0.8 | BLOCK > 0.8. Caught all 10 frauds with zero false positives.
AI Catalyst audit trail documents which model version was used for every scoring run.

Stage 6

Full platform — deploy dashboard and monitor 24/7

What the developer does:

One command deploys the Streamlit fraud dashboard to a live URL on Outerbounds. The fraud ops team sees real-time metrics, tests transactions, and reviews analytics from any browser. Scheduled batch scoring runs hourly via Argo Workflows on K8s. The Outerbounds UI provides run history, cost reports, deployment health, and auto-scaling. AI Catalyst audit trails satisfy compliance reviews.

Live dashboard

4 pages: Dashboard, Test, Analytics, System Status
24/7 at public URL

Scheduled scoring

Hourly batch via Argo
100+ transactions per run
Auto-alerts on BLOCK spikes

Full observability

Outerbounds: runs, costs, deployment health, scaling
AI Catalyst: model audit trails

Customer value: 24/7 fraud protection. Full auditability. Hours to deploy, not weeks.

In our fraud project:

Dashboard deployed at <https://ui-c-271nui.merced.obp.outerbounds.com>
outerbounds app deploy --name fraud-dashboard --public-access --cpu 2 --memory 4096
Auto-scales 1-2 workers. All runs, models, and decisions traceable to specific run IDs.

Full architecture: Anaconda + Outerbounds

From open-source suppliers to production deployment

